Relational Database

1. Intro

1) 정의

Name	Dry/Wet Food	Good Boy (Y/N)
Fido	Dry	Y
Rex	Wet	N
Bubbles	Dry	Y
Cujo	Wet	N

Tag #	Height (in)	Weight (lbs)
1573	15	21
2684	9	7
3795	27	130
4806	6	5

Tag #	Name	Breed	Color	Age
1573	Fido	Beagle	Brown/White	1.5
2684	Rex	Pekingese	White	9
3795	Bubbles	Rottweiler	Black	5
4806	Cujo	Chihuahua	Gold	4

> Data Base는 Data를 어떤 것으로 보느냐에 따라서 다양한 종류의 모델로 나뉠 수 있다.
>
> 그 중에서 Relation model은 1970년 "Ted Codd"가 처음 소개한 모델로, 데이터를 > 데이터간의 관계를 기술한 Table이라고 생각하는 방법이다.

2) 구조

> **Table Name**
>
> 우선 각 Table(=Relation)은 고유의 이름을 갖고 있다.
>
> ---
>
> **Attribute (Column)**

학번
89001
89002
87012
86032

Attribute는 Column 자체를 의미한다.
(위의 Table 에서는 학번, 이름, 학년, 신장, 학과가 Attribute 이다.)

- **Degree(차수)**
  : Relational Database Model에서의 Degree는 Attribute의 개수를 의미한다.

- **Domain**
  : Domain은 실제 Attribute에 입력이 허용되는 값의 집합을 의미한다.
  즉, Attribute의 type이다.

  1. *유한 `Domain`: 만약 위의 Table 에서 모든 학생은 1 학년부터 6 학년까지 있다고 주어졌다면 학년의 Domain 은 1, 2, 3, 4, 5, 6 이 된다.*
  2. *무한 `Domain`: 만약 위의 Table 에서 신장의 값은 100 과 300 사이의 모든 값이 허용된다고 하면 키의 Domain 은 [0, 300] 이 된다.*

- **Domain의 특징**

  1. 모든 Attribute는 Domain을 가진다
  2. Atomic해야한다
     (그 값을 쪼개서 사용할 수 없어야 한다)
  3. 모든 Domain은 null을 허용한다.
     (null은 '값'이 아닌 '상태'라고 생각해야 한다.)

(`Atomic 하지않음`: 만약 위의 Table 에서 이름을 성과 이름으로 나누어 쓸 수 있다고 가정했다면 Atomic 하지 않은 상태인 것이다.) (`Atomic 함`: 위와 반대로 성과 이름을 나눌 수 없다고 가정했다면 Atomic 한 상태가 된다.)

Tuple (Row)

89001	홍길동	2	170	CD

Tuple이란 어떤 값들이 정해진 순서를 가지고 나열된 리스트를 의미한다.

- **Cardinality**
  : Relational Database Model에서의 Cardinality는 Tuple의 개수(Instance의 개수)를 의미한다.

- **N-Tuple**
  : n-Tuple은 원소의 개수(Attribute의 개수)가 n개인 Tuple을 의미한다. : n-Tuple은 `t = <v1, ..., vn>`라고 표기

(`Relation Instance`)
: Table 을 이루는 것은 이 N-Tuple 들의 Set 이다.

> *Tuple들의 집합을 Relation Instance라고 한다.*

2. Relation

1) 정의

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

> **D1 x D2 x ... x Dn 의 부분 집합**
>
> 한 Table이 만약 n개의 Attribute를 가지고 있다고 하자.
> 이 때, Relation이란 몇 N-Tuple들로 이루어진 집합이라고 정의할 수 있다.
>
> *(x, Catesian Product: x는 Catesian Product를 나타내는 연산기호로, 기호의 앞과 뒤의 두 집합에 대해 모든 조합을 구하는 연산기호이다.)*
>
> ---
>
> **특징**
>
> 위에서 알아보았듯이 Database는 다음과 같이 구성된다.
>
> ```
> Attribute의 집합=Tuple
> Tuple의 집합=Relation
> Relation의 집합=Database
> ```
>
> 여기서 Relation의 특징을 확인할 수 있는데 이는 다음과 같다.
>
> - Tuple의 무순서성
> : Tuple은 순서를 가지고 Relation에 참여하지 않는다.
>
> - Attribute의 무순서성
> : Attribute또한 Tuple내에서 순서를 가지지 않는다.

- Attribute의 원자성
  : Attribute는 Multi-Valued값이나, Composite한 값을 가질 수 없다.

- Null값의 활용
  : Attribute의 Domain에는 Null값이 존재할 수 있다.(추후 자세히 설명)

2) Schema & Instance

Schema

ID	name	dept_name	salary

Schema는 구조적인 것, 선언적인것을 의미한다.
C와 같은 프로그래밍 언어에서 흔하게 사용하는 Prototype이랑 비슷한 개념이라고 생각하면 된다.

- **Relation**

```
R(A1, ..., An)
```

- **N-Tuple**

```
t = <v1, v2, ..., vn>
```

- **Attribute**

```
t[Ai]
t.Ai
```

Relation Instance(상태)

Schema와 다르게 Instance는 실제 어떤 값이 입력되어 있는지를 의미한다.

10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

- **표현**
  : instructor의 Instance는 12개의 Tuple을 가지고 있다.

---

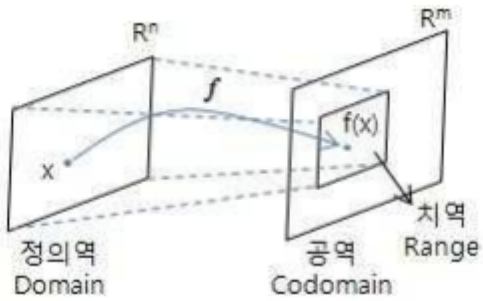
3. Integrity Constraint

`Relational Database의 Schema`는 `Schema` + `Integrity Constraint`로 표현된다.
이때 Integrity Constraint에는 다음과 같이 5종류가 있다.

- *Domain*
  : 각 Attribute에 정의된 Domain을 만족해야함
- *Not Null*
  : Attribute의 값으로 Null값을 허용하지 않음
- *Key*
  : Relation에서 Tuple을 식별하는 Attribute의 집합이 있어야 함
- *Entity Integrity Constraint*
  : 삽입과 수정시 Key와 Not Null을 확인함
- *Referential Integrity Constraint*
  : R에서 S의 Tuple을 참조하는 경우 S에는 해당 Tuple이 반드시 존재해야 함

자세한 내용은 이후부터 살펴보도록 하자.

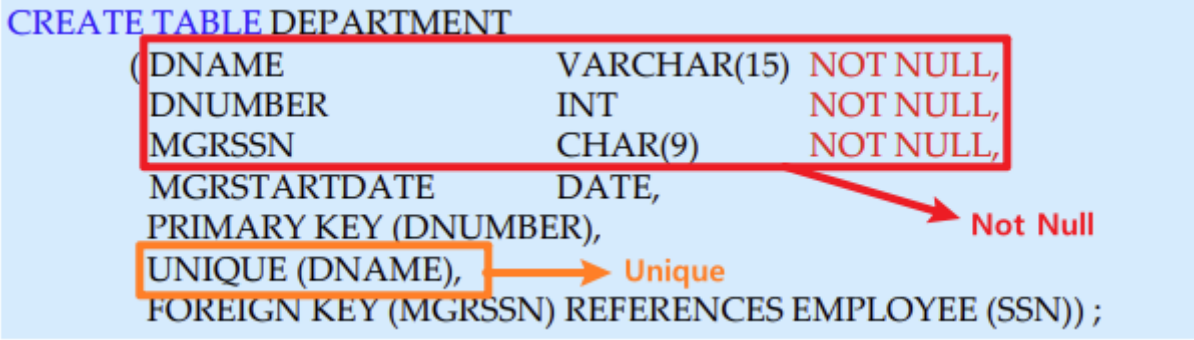
1) Domain



> 문자열

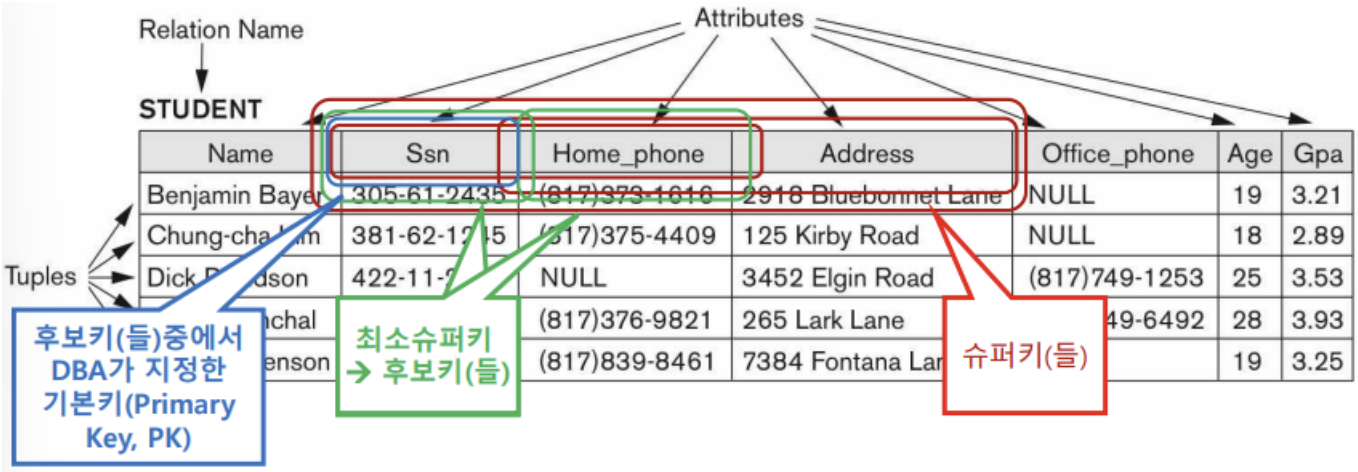
> - 고정길이 문자열
> - 가변길이 문자열
>
> 숫자
>
> - 실수
> - 정수
>
> 기타
>
> - Time ...

2) Not Null



> Attribute의 값으로 Null값을 입력하는 것을 불가능하게 하는 것
>
> 이때 Null의 의미는 여러가지가 있는데 다음 두 가지를 주의해야한다.
>
> - 알려지지 않음(모름)
> : ex. John은 핸드폰이 있으나 그의 핸드폰 번호는 알지 못함
>
> - 아직까지는 이용할 수 없음(보류)
> : ex. John은 자신의 번호를 의도적으로 적지 않음
>
> - 적용할 수 없음(정의되지 않음)
> : ex. college_degree는 대학 미 졸업자에게는 정의되지 않음
>
> ---
>
> **Unique**
>
> 해당 Attribute에 대한 중복된 값이 존재하는 것을 불가능하게 하는 것

3) Key



> Relation에서 Tuple을 식별하는 Attribute의 집합이 있어야 함
>
> ---
>
> 1. **Super Key**
>
> Attribute의 부분 집합 중, 해당 Domain의 모든 값이 Unique하도록 하는 부분집합을 의미한다.
>
> > - Not Null
> > - 유일성(Uniqueness): 만족
> > - 최소성(Minimality): 상관없음
>
> 즉, 위의 그림에서 각 Attribute를 SuperKey는 다음과 같다.
>
> Ssn은 Super Key다.
> {Home_Phone, Age}는 Super Key다.
> Name은 Super Key가 아니다.
>
> *(만약 위의 Table에서 같은 이름의 학생을 받지 않는다는 규칙이 존재한다면, name도 Super Key가 될 수 있다.)*
>
> ---
>
> 2. **Candidate Key**
>
> Super Key중에서 어떠한 부분 집합도 Super Key가 될 수 없을 때 해당 Key를 Candidate Key라고 한다.
>
> > - Not Null
> > - 유일성(Uniqueness): 만족
> > - 최소성(Minimality): 만족
>
> 즉, 위의 그림에서 각 Attribute를 SuperKey는 다음과 같다.
>
> Ssn은 Candidate Key이다.
> Home_Phone은 Candidate Key이다.
> {Home_Phone, Age}는 Candidate Key가 아니다..
>
> ---
>
> 3. **Primary Key**
>
> Candidate Key중에서 DB설계자가 Table안에서 튜플을 구별하기 위한 수단으로 선택한 Key를 Primary Key라고 한다.

> - Not Null
> - 유일성(Uniqueness): 만족
> - 최소성(Minimality): 만족
> - 선택받은 Attribute Set

즉, 위의 그림에서 DB설계자가 Ssn을 Primary Key로 지정했다면 Primary Key는 오직 Ssn 뿐이다.

중요: *따라서, Primary Key를 설정할 때에는 앞으로도 그 값이 절대로 변하지 않는다는 것을 가정해야 한다.*

(Schema: 일반적으로 Relation Schema에서 다른 Attribute보다 Primary Key를 먼저 나열하고, 밑줄을 쳐서 표시한다.)

---

4. **Alternate Key**

Candidate Key중에서 DB설계자의 선택을 받지 못한 나머지 키들을 의미한다.

4) Referential Integrity

> A의 한 Attriubute가 B를 참조할 경우 해당 Attribute에 등장하는 값은 반드시 B의 Attribute에서 적어도 하나의 Domain으로 출현해야 한다.
>
> ---
>
> **Foreign Key**
>
> 참조하는 Attribute의 값이 해당 Table(=Relation)에서 Primary Key로 사용되고 있을 경우 참조 당하는 Table(=Relation)에서는 Foreign Key라고 부른다.
>
> ---
>
> **주의점 1**
>
> - Foreign Key를 받는 Attribute는 Null일 수 있다.
> - Foreing Key를 주는 Attribute는 Null일 수 없다.
> (해당 Table에서 Primary Key의 역할을 하기 때문이다.)
>
> 즉, 이 경우의 Null의 의미는 "모름"과 "보류"이다.
> ("정의되지 않음"의 의미가 아니라는 점을 유의하자)
>
> ---
>
> **주의점 2**
>
> 삽입(Insert)과 삭제(Delete)시 부모(Foreign Key를 주는 Table)와 자식(Foreign Key를 받는 Table)간의 실행 순서에 주의해야 한ㄷ,

5) Entity integrity

> Table(=Relation)은 반드시 기본키를 가져야 하고 그에 따른 Key Integrity를 항상 지켜야 한다.